



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.food_positions` (locations of food objects)
`self.opponents` (locations of dynamic entities)

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The existence of `self.opponents` (other active entities that move dynamically and affect the agent's score upon collision) means the state transitions depend on the behavior of other agents, making it a multi-agent environment.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of everything the agent has perceived/sensed from its environment from the start of its operation up to the current time step.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The dictionary provides full global spatial visibility of `opponent_positions` and `agent_pos`, but the food detection is local/boolean (`smells_food` only checks if food is under the agent's current cell, rather than giving a full map of food locations). Additionally, full grid/wall layout is not explicitly broadcast in

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure (P)
(specifically deducting 5 points from `self.score`)

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Measures must track actual achievements in the environment (e.g., collecting food, avoiding walls) rather than internal compute time, preventing agents from "cheating" by looping computational steps without solving the task.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

- Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?**

If `self.toxic_traps` exist in the state but are missing from `get_percept()`, the environment becomes Partially Observable (or unobservable with respect to traps) because the agent's sensors cannot detect critical aspects of the physical

Step 2.2: Updating the Perception Subsystem (`get_percept`)

- Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos)` in `self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Rationality means taking actions that maximize expected performance given the current percept sequence and built-in knowledge. Adding 'smells_toxin' informs the agent whether its current square contains a hazard, allowing it to incorporate this information to choose actions that avoid severe penalties. (15 points)

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World Exploit occurred when the performance metric rewarded the agent 1 point every time it cleaned dirt. A "rational" agent under this metric realized it could dump dirt back onto the floor and immediately sweep

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING